

U.T. 5. Interacción con el usuario: eventos y formularios.

2.1. Modelo de gestión de eventos

Los **eventos** son mecanismo que se accionan cuando el usuario realiza un cambio sobre una página web.

El encargado de crear la jerarquía de objetos que compone una página web es el DOM (DocumentObjectModel). Por tanto es el **DOM** el encargado de gestionar los eventos.

Para poder controlar un evento se necesita un **manejador**. El manejador es la palabra reservada que indica la acción que va a manejar. En el caso del evento *click*, el manejador sería `onClick`.

Ejemplo:

```
<IMG SRC="mundo.jpg" onclick="alert('Click en imagen');">
```

El ejemplo anterior se puede realizar de otro modo llamando a una función:

```
<html>
<head>
<title>Paginade Evento</title>
<script>
function func1() {
alert("Click en imagen");
}
</script>
</head>
<body>
<IMG SRC="mundo.jpg" onclick="func1();">
</body>
</html>
```

La especificación DOM define cuatro grupos de eventos dividiéndolos según su origen:

- Eventos del ratón.

- Eventos del teclado.
- Eventos HTML.
- Eventos DOM.

2.1.1. Eventos del ratón

Click. Este evento se produce cuando pulsamos sobre el botón izquierdo del ratón. El manejador de este evento es onclick.

Dblclick. Este evento se acciona cuando hacemos un doble click sobre el botón izquierdo del ratón. El manejador de este evento es ondblclick.

Mousedown. Este evento se produce cuando pulsamos un botón del ratón. El manejador de este evento es onmousedown.

Mouseout. Este evento se produce cuando el puntero del ratón esta dentro de un elemento y este puntero es desplazado fuera del elemento. El manejador de este evento es onmouseout.

Mouseover. Este evento al revés que el anterior se produce cuando el puntero del ratón se encuentra fuera de un elemento, y este se desplaza hacia el interior. El manejador de este evento es onmouseover.

Mouseup. Este evento se produce cuando soltamos un botón del ratón que previamente teníamos pulsado. El manejador de este evento es onmouseup.

Mousemove. Se produce cuando el puntero del ratón se encuentra dentro de un elemento. Es importante señalar que este evento se producirá continuamente una vez tras otra mientras el puntero del ratón permanezca dentro del elemento. El manejador de este evento es onmousemove.

2.1.2. Eventos del teclado

Keydown. Este evento se produce cuando pulsamos una tecla del teclado. Si mantenemos pulsada una tecla de forma continua, el evento se produce una y otra vez hasta que soltemos la misma. El manejador de este evento es onkeydown.

KeyPress. Este evento se produce si pulsamos una tecla de un carácter alfanumérico (El evento no se produce si pulsamos enter, la barra espaciadora,

etc...). En el caso de mantener una tecla pulsada, el evento se produce de forma continuada. El manejador de este evento es `onkeypress`.

Keyup. Este evento se produce cuando soltamos una tecla. El manejador de este evento es `onkeyup`.

2.1.3. Evento HTML

Load. El evento *load* hace referencia a la carga de distintas partes de la página. Este se produce en el objeto Window cuando la página se ha cargado por completo. En el elemento `<img>` actúa cuando la imagen se ha cargado. En el elemento `<object>` se acciona al cargar el objeto completo. El manejador es `onload`.

Unload. El evento *unload* actúa sobre el objeto Window cuando la pagina ha desaparecido por completo (por ejemplo, si pulsamos el aspa cerrando la ventana del navegador). También se acciona en el elemento `<object>` cuando desaparece el objeto. El manejador es `onunload`.

Abort. Este evento se produce cuando el usuario detiene la descarga de un elemento antes de que haya terminado, actúa sobre un elemento `<object>`. El manejador es `onabort`.

Error. El evento *error* se produce en el objeto Window cuando se ha producido un error en JavaScript. En el elemento `<img>` cuando la imagen no se ha podido cargar por completo y en el elemento `<object>` en el caso de que un elemento no se haya cargado correctamente. El manejador es `onerror`.

Select. Se acciona cuando seleccionamos texto de los cuadros de textos `<input>` y `<textarea>`. El manejador es `onselect`.

Change. Este evento se produce cuando los cuadros de texto `<input>` y `<textarea>` pierden el foco y el contenido que tenían ha variado. También se producen cuando un elemento `<select>` cambia de valor. El manejador es `onchange`.

Submit. Este evento se produce cuando pulsamos sobre un botón de tipo submit. El manejador es `onsubmit`.

Reset. Este evento se produce cuando pulsamos sobre un botón de tipo reset. El manejador es `onreset`.

Resize. Este evento se produce cuando redimensionamos el navegador, actúa sobre el objeto Window. El manejador es onresize.

Scroll. Se produce cuando varía la posición de la barra de scroll en cualquier elemento que la tenga. El manejador es onscroll.

Focus. Este evento se produce cuando un elemento obtiene el foco. El manejador es onfocus.

Blur. Este evento se produce cuando un elemento pierde el foco. El manejador es onblur.

2.1.4. Evento DOM

DOMSubtreeModified. Este evento se produce cuando añadimos o eliminamos nodos en el subárbol de un elemento o documento.

DOMNodeInserted. Este evento se produce cuando añadimos un nodo hijo a un nodo padre.

DOMNodeRemoved. Este evento se produce cuando eliminamos un nodo que tiene nodo padre.

DOMNodeRemovedFromDocument. Este evento se produce cuando eliminamos un nodo del documento.

DOMNodeInsertedIntoDocument. Este evento se produce cuando añadimos un nodo al documento.

2.2. Utilización de formularios desde código

Un **formulario** web sirve para enviar, tratar y recuperar datos que son enviados y recibidos entre un cliente y un servidor web.

Cada elemento del formulario almacena un tipo de dato o acciona una de sus funcionalidades.

Los formularios disponen de una arquitectura, en este contexto están enmarcados en el lenguaje HTML.

2.2.1. Estructura de un formulario

Los formularios se definen con etiquetas.

La etiqueta principal es **<form> </form>**.

Para que sea funcional, la etiqueta **<form>** necesita inicializar dos atributos:

- **action**: Contiene la URL donde se redirigen los datos del formulario.
- **method**: Indica el método por el cual el formulario envía los datos. Puede ser **POST** o **GET**.

Ejemplo:

```
<html>
<head>
<title>Ejemplode formulario</title>
</head>
<body>
<h3>Formulario</h3>
<form action="mipagina.php" method="post">
...
</form>
</body>
</html>
```

2.2.2. Elementos de un formulario

El elemento principal del formulario se denomina con la etiqueta **<input>**.

2.2.3. Estructura de una etiqueta input

Atributos de la etiqueta input:

Type. Indica el tipo de elemento que vamos a definir. De él dependen el resto de parámetros. Los valores posibles que acepta el atributo type son:

- Text (cuadro de texto).
- Password (cuadro de contraseña, los caracteres aparece ocultos tras asteriscos).
- Checkbox (casilla de verificación).
- radio (opción de entre dos o más).

- Submit (botón de envío del formulario).
- Reset (botón de vaciado de campos).
- file (botón para buscar ficheros).
- Hidden (campo oculto para, el usuario no lo visualiza en el formulario)
- Image (botón de imagen en el formulario).
- Button (botón del formulario).

Name. El atributo name asigna un nombre al elemento. Si no le asignamos nombre a un elemento, el servidor no podrá identificarlo y por tanto no podrá tener acceso al elemento.

Value. El atributo value inicializa el valor del elemento. Los valores dependerán del tipo de dato, en ocasiones los posibles valores a tomar serán: verdadero o falso.

Size. Este atributo asigna el tamaño inicial del elemento. El tamaño se indica en pixeles. En los campos text y password hace referencia al número de caracteres.

Maxlength. Este atributo indica el número máximo de caracteres que pueden contener los elementos text y password. Es conveniente saber que el tamaño de los campos text y password es independiente del número de caracteres que acepta el campo.

Checked. Este atributo es exclusivo de los elementos checkbox y radio. En el definimos que opción por defecto queremos seleccionar.

Disable. Este atributo hace que el elemento aparezca deshabilitado. En este caso el dato no se envía al servidor.

Readonly. Este atributo sirve para bloquear el contenido del control, por tanto el valor del elemento no se podrá modificar.

Src. Este atributo es exclusivo para asignar una URL a una imagen que ha sido establecida como botón del formulario.

Alt. El atributo alt, incluye una pequeña descripción del elemento. Habitualmente y si no lo hemos desactivado cuando posicionamos el ratón (sin pulsar ningún botón) encima del elemento, podemos visualizar la descripción del mismo.

2.2.4. Tipos de input

Cuadro de texto:

Este input muestra un cuadro de texto vacío en el que el usuario puede introducir un texto.

Este es uno de los elementos más usados. La forma de indicar que es un campo de texto es: `type="text"`

Cuadro de contraseña:

El cuadro de contraseña es como el cuadro de texto, con la diferencia que los caracteres que escribe el usuario no se ven en pantalla.

En su lugar los navegadores muestran asteriscos o puntos.

```
<input type="password" name="contrasenia" />
```

Casilla de verificación:

Estos elementos permiten al usuario activar o desactivar la selección de cada una de las casillas de forma individual.

```
<p>Colores favoritos</p> </br>
<input name="rojo" type="checkbox" value="ro"/> Rojo </br>
<input name="azul" type="checkbox" value="az"/> Azul </br>
<input name="verde" type="checkbox" value="ve"/> Verde
```

Opción de radio:

Este tipo de elemento agrupa una serie de opciones excluyentes entre sí. De esta forma el usuario sólo puede coger una opción de entre todas las que tiene establecidas un grupo de botones radio.

Género

```
</br><input type="radio" name="género" value="M"> Hombre  
</br><input type="radio" name="género" value="F"> Mujer
```

Botón de envío:

Este elemento es el encargado de enviar los datos del formulario al servidor. En este caso el type toma el valor submit. El valor del atributo value se mostrará en este caso en el botón generado.

```
<input type="submit" name="enviar" value="Enviar">
```

Botón de reset:

Este elemento es un botón que establece el formulario a su estado original.

Ficheros adjuntos:

Este tipo de input permite adjuntar ficheros adjuntos. El elemento añade de forma automática un cuadro de texto que se dispondrá para almacenar la dirección del fichero adjunto seleccionado.

```
Fichero adjunto <input type="file" name="fichero"/>
```

Campos ocultos:

Los campos ocultos no son visibles en el formulario por el usuario. Estos elementos son útiles para enviar información de forma oculta que no tenga que ser tratada por el usuario.

```
<input type="hidden" name="campoOculto" value="cambiar"/>
```

Botón de imagen:

Este elemento es una personalización de un botón, cambiando el aspecto por defecto que tienen los botones de un formulario por una imagen.

```
<input type="image" name="enviar" src="imagen_mundo.jpg"/>
```

Botón:

Existe un elemento botón, al que podemos asociar diferentes funcionalidades. De esta forma no nos tenemos que ceñir los botones de submit o reset que nos ofrecen los formularios.

```
<input type="button" name="opcion" value="Opcionvalidar"/>
```

Ejemplo Completo

```
<form action="pagina.php" method="post" enctype="multipart/form-data"
/>
<br/>
Nombre:<input type="text" name="nombre" value="" size="42"
maxlength="30"/>
Apellidos:<input type="text" name="ape" value="" size="40"
maxlength="80"/>
DNI:<input type="text" name="dni" value="" size="10" maxlength="9" />
Sexo:<input type="radio" name="sexo" value="hombre"
checked="checked"/>Hombre
<input type="radio" name="sexo" value="mujer" />Mujer
Incluir mi foto: <input type="file" name="foto" /> <br/>
<input name="publ" type="checkbox" value="publicidad"
checked="checked"/> Enviar publicidad
<input type="submit" name="enviar" value="Guardar cambios" />
<input type="reset" name="limpiar" value="Borrar los datos
introducidos"/>
</form>
```

Nombre:

Apellidos:

DNI:

Sexo:

☒ Hombre

☐ Mujer

Incluir mi foto:

☒ Enviar publicidad

2.3. Modificación de apariencia y comportamiento

2.3.1. Modificación de la apariencia de un formulario

HTML / CSS

Fieldset y Legend

La etiqueta “fieldset” suele usarse principalmente en formularios para poder agrupar campos que están relacionados entre sí. Aunque también podemos utilizarlo para agrupar una serie de definiciones o de textos relacionados entre sí.

```
<html>
<body>
<form>
  <fieldset>
    <legend>Datos Personales</legend>
    Nombre: <input type="text"><br>
    Email: <input type="text"><br>
  </fieldset>
</form>
</body>
</html>
```

2.3.2. Modificación del comportamiento de un formulario

Los formularios tienen unas acciones predeterminadas por defecto. Sin embargo es posible darle otro comportamiento a uno de sus elementos. Por ejemplo podríamos querer que los datos se envíen a URLs diferentes en base a un dato que introduzca el usuario

```
<script language="javascript">
function enviar(form) {
if (formulario.alta.checked== true) {
formulario.action= "paginas/alta.html";
}
if (formulario.alta.checked== false) {
formulario.action= "paginas/baja.html";
}
```

```
}  
form.submit()  
}  
</script>
```

2.4. Validación y envío

El usuario puede cometer errores al rellenar un formulario.

Si por ejemplo se espera un código postal y se introduce el nombre de una ciudad, se producirá un error.

Para controlar estas situaciones se pueden usar las validaciones.

Este tipo de validaciones se suelen realizar llamando a una función que analice si el dato cumple con las restricciones establecidas

2.4.1. Estructura del form para validar datos

Para que el formulario detecte que debe realizar una validación antes de enviar los datos, debemos indicarlo en su estructura.

```
<form action="URL" method="post" name="formValidado" onsubmit="return  
validar()">...</form>
```

Validar un campo como obligatorio:

```
<script type="text/javascript">  
function validacion() {  
    valor = document.getElementById("campo").value;  
    if( valor == null || valor.length== 0 ){  
        alert("El campo no puede ser vacío");  
        return false;  
    }  
    return true;  
}  
</script>
```

Validar un campo de texto como numérico:

```
<script type="text/javascript">
```

```
function validaNum() {  
valor = document.getElementById("telefono").value;  
if( isNaN(valor) ) {  
    alert("El campo tiene que ser numérico");  
    return false;  
}  
return true;  
}  
</script>
```

Validar si una fecha es correcta:

```
<script type="text/javascript">  
function validaFecha() {  
vardia= document.getElementById("dia").value;  
varmes = document.getElementById("mes").value;  
varano = document.getElementById("ano").value;  
fecha = new Date(ano, mes, dia);  
if( !isNaN(fecha) ) {  
return false;  
}  
Return true;  
}  
</script>
```

Validar un checkbox:

```
<script type="text/javascript">  
function validaCheck() {  
elemento= document.getElementById("campoCondiciones");  
if( !elemento.checked) {  
    return false;  
}  
return true;  
}  
</script>
```

2.5. Expresiones regulares

Las **expresiones regulares** describen un conjunto de elementos que siguen un patrón.

Un ejemplo podría ser todas las palabras que comienzan por la letra “a” minúscula.

JavaScript implementa expresiones regulares y facilita las comprobaciones de ciertos datos que deben seguir una estructura concreta.

2.5.1. Caracteres especiales de las expresiones regulares

^ Principio de entrada o línea. Este carácter indica que las cadenas deberán comenzar por el siguiente carácter. Si este fuera una “a” minúscula como indicamos en el punto anterior la expresión regular sería, ^a.

\$ Fin de entrada o línea. Indica que la cadena debe terminar por el elemento precedido al dólar.

*** El carácter anterior 0 o más veces.** El asterisco indica que el carácter anterior se puede repetir en la cadena 0 o más veces.

+ El carácter anterior 1 o más veces. El símbolo más indica que el carácter anterior se puede repetir en la cadena una o más veces.

? El carácter anterior una vez como máximo. El símbolo interrogación indica que el carácter anterior se puede repetir en la cadena cero o una vez

. **Cualquier carácter individual.** El símbolo punto indica que puede haber cualquier carácter individual salvo el de salto de línea.

x|yx ó y:La barra vertical indica que puede ser el carácter x o el y.

{n} n veces el carácter anterior. El carácter anterior a las llaves tiene que aparecer exactamente n veces.

{n,m} Entre n y m veces el carácter anterior.El carácter anterior a las llaves tiene que aparecer como mínimo n y como máximo m veces.

[abc] Cualquier carácter de los corchetes. En la cadena puede aparecer cualquier carácter que este incluido en los corchetes.

[^abc] Un carácter que no esté en los corchetes. En la cadena pueden aparecer todos los caracteres que no estén incluidos en los corchetes.

\b Fin de palabra. Este símbolo indica que tiene que haber un fin de palabra o retorno de carro.

\B No fin de palabra. El símbolo \B indica cualquiera que no sea un límite de palabra.

\d Cualquier carácter dígito. Este símbolo indica que puede haber cualquier carácter numérico, de 0 a 9.

\D Carácter que no es dígito. Este símbolo indica que puede haber cualquier carácter siempre que no sea numérico.

\f Salto de página. Este símbolo indica que tiene que haber un salto de página

\n Salto de línea. Este símbolo indica que tiene que haber un salto de línea.

\r Retorno de carro. Este símbolo indica que tiene que haber un retorno de carro.

\s Cualquier espacio en blanco. Este símbolo indica que tiene que haber un carácter individual de espacio en blanco: espacios, tabulaciones, saltos de página o saltos de línea.

\S Carácter que no sea blanco. Este símbolo indica que tiene que haber cualquier carácter individual que no sea un espacio en blanco.

\t Tabulación. Este símbolo indica que tiene que haber cualquier tabulación.

\w Carácter alfanumérico. Este símbolo indica que puede haber cualquier carácter alfanumérico.

\W Carácter que no sea alfanumérico. Este símbolo indica que puede haber cualquier carácter que no sea alfanumérico.

2.5.2. Validar un formulario con expresiones regulares

Combinando las anteriores expresiones se puede abordar una infinidad de **patrones** para validar datos en los formularios.

Se pueden validar por ejemplo campos como:

- Correo electrónico.
- Teléfono.
- Código postal.
- DNI.
- Etc.

Validar una dirección de correo electrónico:

```
<script type="text/javascript">
function validaEmail() {
valor = document.getElementById("campo").value;
if(!(/^\w+([-+.']\w+)*@\w+([-.\w+)*\.\w+([-.\w+)]\w+)/.test(valor)) ) {
return false;
}
return true;
}
</script>
```

Validar un DNI:

```
<script type="text/javascript">
function validaDNI(){
valor = document.getElementById("dni").value;
varletras = ['T','R','W','A','G','M','Y','F','P','D','X',
'B','N','J','Z','S','Q','V','H','L','C','K','E','T'];
if( !(/^^\d{8}[A-Z]$/ .test(valor)) ){ return false; }
if(valor.charAt(8) != letras[(valor.substring(0, 8))%23])
{ return false; }
return true;
}
</script>
```

Validar un número de teléfono:

```
function validaTelefono(){
valor = document.getElementById("telefono").value;
if( !(/^^\d{9}$/ .test(valor)) ) {
return false;
}
return true;
}
```

2.6. Utilización de cookies

Las **cookies** surgieron como necesidad ante algunas ausencias tecnológicas del protocolo HTTP.

Una **cookie** es un fichero de texto que se almacena en el ordenador del cliente.

Son un fichero propio de cada navegador.

Surgieron con el fin de mantener la información de los carritos de compra virtuales a través de la web.

2.6.1. Usos de las cookies

Mantener opciones de visualización:

- Las *cookies* son utilizadas en ocasiones para mantener unas preferencias de visualización.
- Algunas páginas como *google*, permiten que el usuario haga una configuración de su página de entrada en el buscador, a través de las *cookies* el servidor reconoce ciertos aspectos que el usuario configuró y conserva el aspecto.

Almacenar variables:

- El servidor puede utilizar las cookies para almacenar variables que se necesiten utilizar en el navegador.
- Un ejemplo sería, una página en la que nos solicitan unos datos, en la siguiente nos solicitan otros datos y así hasta la página final. Los datos de las páginas anteriores se irán almacenando en las cookies hasta que se finaliza el ciclo del formulario y el usuario envía los datos al servidor.
- Antes del envío al servidor se recuperarán todos los campos del formulario que están guardados en las cookies.

Realizar un seguimiento de la actividad del usuario:

- En ocasiones los servidores hacen uso de las cookies para almacenar ciertas preferencias y hábitos que el usuario tiene a la hora de navegar.
- Con esta información, el servidor personaliza sus servicios y publicidad orientándolo a cada cliente en particular.
- Estos fines no son del todo lícitos si la entidad que realiza estas actividades no avisa al usuario de que está realizando estas acciones.

Autenticación:

- Autenticar a los usuarios es uno de los usos más habituales de las cookies.

- A través de las cookies, el navegador guarda los datos del usuario, al realizar una petición al servidor, el navegador envía las cookies junto con la petición.
- Las cookies tienen una caducidad, cuando pasa un periodo de tiempo establecido, estas desaparecen junto con el fichero de texto que guarda el navegador.
- Habitualmente, como mecanismo de seguridad, las aplicaciones Web aplican un tiempo máximo de inactividad, por ejemplo de 15 minutos, tras el cual las cookies caducan, si no se produjo movimiento en la navegación de la aplicación web.

2.6.2. Lectura y escritura de las cookies

Los dos procesos de implementación principales de una *cookie* son la escritura y la lectura de la misma.

A continuación se presentan tres funciones que sirven para:

- Devolver el valor de una cookie.
- Escribir una cookie.
- Comprobar si existe un valor para la cookie

```
<html>
<head>
<script type="text/javascript">
function getCookie(c_name){
var i, x, y, ARRcookies=document.cookie.split(";");
for (i=0; i<ARRcookies.length; i++) {
    x=ARRcookies[i].substr(0,ARRcookies[i].indexOf("="));
    y=ARRcookies[i].substr(ARRcookies[i].indexOf("=")+1);
    x=x.replace(/^\s+|\s+$/g,"");
    if (x==c_name){
        return unescape(y);
    }
}
}
function setCookie(c_name,value,exdays){
varexdate=new Date();
```

```
exdate.setDate(exdate.getDate() + exdays);
varc_value=escape(value)+(exdays==null)?"":";
expires="+exdate.toUTCString());
document.cookie=c_name+ "=" + c_value;
}
function checkCookie(){
varusername=getCookie("username");
if (username!=null && username!=""){
    alert("Bienvenido" + username);
}else{
    username=prompt("Por favor, Introduzca su usuario:", "");
    if (username!=null && username!=""){
        setCookie("username",username,365);
    }
}
}
}
</script>
</head>
<body><input type="button" name="chequeaCookie"
value="Chequear las cookies" onclick="checkCookie();">
</body>
</html>
```

http://librosweb.es/ajax/capitulo_6.html

http://librosweb.es/javascript/capitulo_7.html